



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ

Rai Technology University

(Estb. under Karnataka Act. No. 40 of 2013)

ASSIGNMENT NO:02

Subject Title: Deep Learning

Subject Code:OCS351T

Submitted By:

Student Name: Sathish Gowda R S

University Enrolment Number:RTU24101CS011

Program Name:B.tech (CSE)

Semester: 5TH SEM

Session: 2026

Submitted To

Faculty Name: Mr.Satyam Sir

**Department of Computer Science &Engineering
College of Engineering**

Rai Technology University Bengaluru-561204

Deep Learning

Introduction

Deep learning is a branch of machine learning that uses artificial neural networks to learn patterns from data. It is widely used in image classification, speech recognition, natural language processing and many other applications. In this assignment a simple neural network is developed using tensorflow & keras to classify handwritten digits from 0 to 9.

Objectives

- Load and explore the MNIST handwritten digit dataset
- Display sample handwritten digit images
- Design, compile and train a simple neural network using TensorFlow/Keras.
- Evaluate the trained model using test accuracy
- Visualize training and validation accuracy & loss
- Test the model on five handwritten digit images and compare actual & predicted labels.

Dataset Description

MNIST is a standard dataset used for handwritten digit classification. It contains 60,000 training images and 10,000 testing images and 10,000 testing images. Each image represents one of the ten digits, 0 through 9.

Dataset shapes using in the program:

- Training images : (60000, 28, 28)
- Training labels : (60000,)
- Testing images : (10000, 28, 28)
- Testing labels : (10000,)

Tools and Technologies

- Python programming language
- TensorFlow/Keras for building & training neural network
- Numpy for numerical operations
- Matplotlib for displaying images & plotting graphs
- Google Colab for executing program

Methodology

The workflow begins by loading the MNIST dataset using TensorFlow built-in Keras dataset function. Sample images are displayed to understand the data. The image pixels are then normalized from 0-255 to 0-1.

A sequential neural network is created. The first layer is flatten which converts each 28×28 image into a one-dimensional vector of 784 values. A dense layer with 128 neurons & ReLU activation is then used to learn useful patterns.

The model is compiled using the Adam optimizer, sparse categorical cross entropy loss function & accuracy as the evaluation metric. It is trained for five epochs with 10% of the training data used for validation.

Neural network Architecture

<u>Layer</u>	<u>Output</u>	<u>Activation</u>	<u>Purpose</u>
Flatten	784	None	Convert 28×28 image to vector.
Dense	128	ReLU	Learn image features
Dense	10	Softmax	Classify digits 0-9

Results & Evaluation

After training the model is evaluated on the 10,000 test images. The test accuracy printed by the program should be recorded below from the actual Colab output.

Metric

Test accuracy

Training Epochs

validation split

optimizer

Loss function

Result

5

10-1.

Adam

Spate categorical cross entropy

Prediction on five images

Five images from MNIST test set are passed through the trained model. The model returns probabilities for all ten digit classes, and the class with the highest probability is selected as the predicted label.

Image

Image 1

Image 2

Image 3

Image 4

Image 5

Actual label

—

—

—

—

—

Predicted label

—

—

—

—

—

Experiment and Comparison

one experiment was performed by changing the number of neurons in hidden dense layer from 128 to 64. The modified model was trained using the same dataset, optimizer, loss function & number of epochs. Its test accuracy was then compared with original model.

model

original model - 128 neurons

modified model - 64 neurons

Test Accuracy

—

—

Conclusion

In this assignment, a simple neural network was successfully designed using TensorFlow/Keras for handwritten digit classification. The MNIST dataset was loaded & explored. Sample images were visualized and the input pixels were normalized. A neural network containing a flatten layer, a 128-neuron ReLU hidden layer and a 10 neuron softmax output layer was trained for five epochs.

The trained model was evaluated using test accuracy & training/validation accuracy & loss were visualized. Five test images were also classified and their actual & predicted labels were compared.

Future Scope

- use Convolutional neural networks (CNNs) for better image feature extraction.
- Experiment with different numbers of layers, neurons, epochs & dropout.
- use data augmentation to improve generalization.
- Deploy the trained model as a web or mobile application for digit recognition.

References

1. TensorFlow/Keras documentation - MNIST dataset & neural network APIs
2. MNIST handwritten digit dataset
3. Python Numpy and matplotlib documentation.

Import tensorflow as tf

Import numpy as np

Import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

Print ("Training images :", x_train.shape)

Print ("Testing images :", x_test.shape).

DATASET

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 0o 0un/step
Training images: (60000, 28, 28)
Testing images: (10000, 28, 28)

plt.figure(figsize=(10,4))

for i in range(10):

plt.subplot(2,5,i+1)

plt.imshow(x_train[i].cmap="gray")

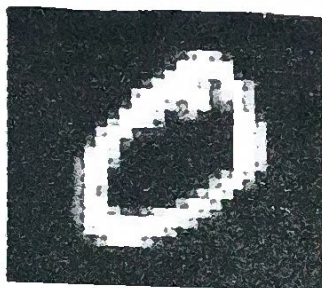
plt.title("Label: " + str(y_train[i]))

plt.axis("off")

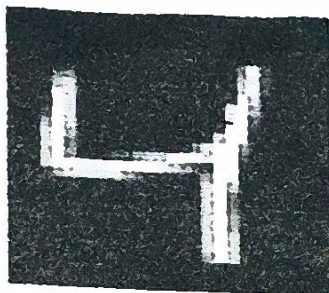
plt.show()



Label: 2



Label: 1



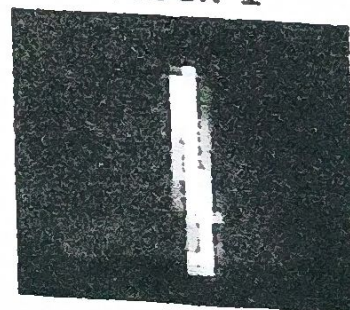
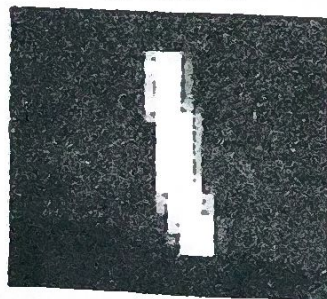
Label: 3



Label: 1



Label: 4



x_train = x_train/255.0

x_test = x_test/255.0.

```

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])

```

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
history = model.fit(
    x_train, y_train,
    epochs=5,
    validation_split=0.1)

```

EXPERIMENT -- MODIFIED MODEL (64 NEURONS)					
Epoch 1/5	1688/1688	8s	5ms/step	accuracy: 0.9117	loss: 0.3159
Epoch 2/5	1688/1688	6s	4ms/step	accuracy: 0.9556	loss: 0.1498
Epoch 3/5	1688/1688	7s	4ms/step	accuracy: 0.9675	loss: 0.1088
Epoch 4/5	1688/1688	6s	4ms/step	accuracy: 0.9729	loss: 0.0875
Epoch 5/5	1688/1688	7s	4ms/step	accuracy: 0.9786	loss: 0.0714

```

test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_accuracy)

```

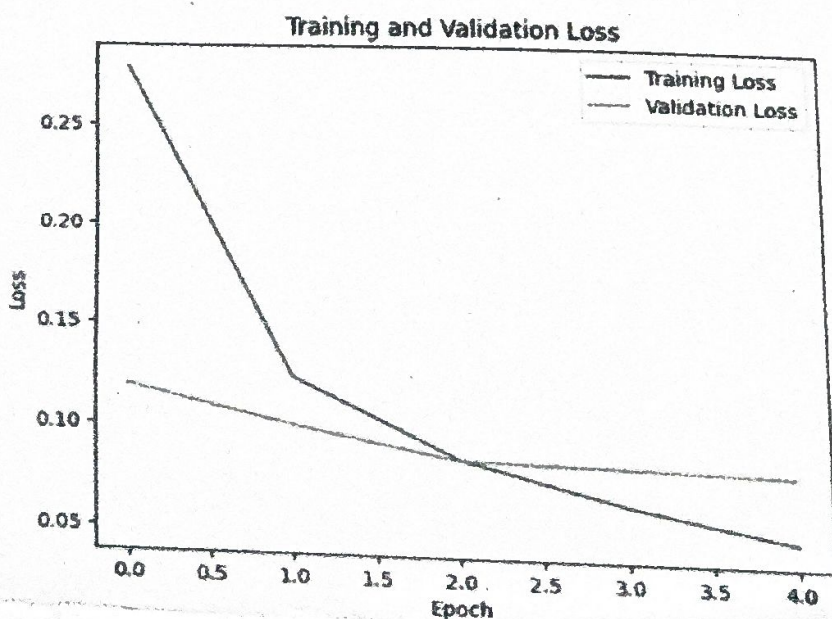
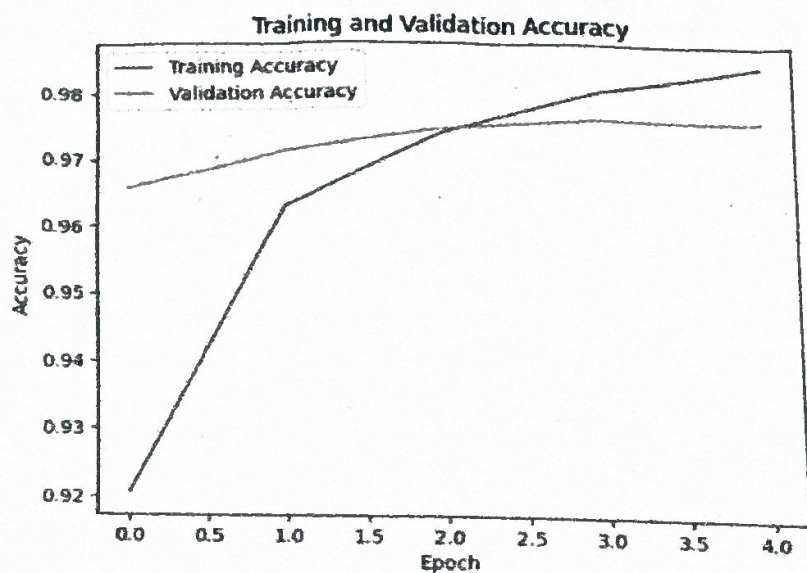
TEST EVALUATION			
313/313	1s	3ms/step	accuracy: 0.9738 loss: 0.0869
Test Accuracy: 0.973800003528595			

```

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Training and Validation Accuracy")
plt.show()

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("Training and Validation Loss")
plt.show()

```



```

Predictions = model.predict(x_test[:5])
Predicted_labels = np.argmax(Predictions, axis=-1)

```

```

plt.figure(figsize=(10,2))

```

```

for i in range(5):

```

```

    plt.subplot(1, 5, i+1)

```

```

    plt.imshow(x_test[i], cmap='gray')

```

```

    plt.title(f"Actual: {x_test[i]} \n Predicted: {Predicted_labels[i]}")

```

```

    plt.show()

```

```

    print("Actual labels: ", x_test[:5])

```

```

    print("Predicted labels: ", Predicted_labels)

```

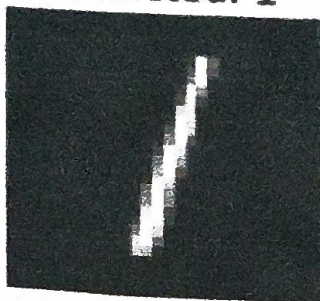
Actual: 7
Predicted: 7



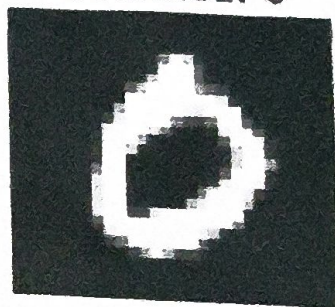
Actual: 2
Predicted: 2



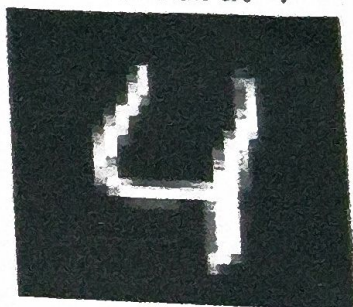
Actual: 1
Predicted: 1



Actual: 0
Predicted: 0



Actual: 4
Predicted: 4



Actual Labels: [7 2 1 0 4]
Predicted Labels: [7 2 1 0 4]

```

model2 = tf.keras.Sequential([

```

```

    tf.keras.layers.Flatten(input_shape=(28,28)),

```

```

    tf.keras.layers.Dense(64, activation='relu'),

```

```

    tf.keras.layers.Dense(10, activation='softmax'),

```

```

])

```

```

model2.compile(

```

```

    optimizer='adam',

```

```

    loss='sparse_categorical_crossentropy',

```

```

    metrics=['accuracy'])

```

history2 = model2.fit(

x_train, y_train,

epochs=5,

validation_split=0.1,

verbose=1

)

test_loss2, test_accuracy2 = model2.evaluate(x_test, y_test,
verbose=0)

Print ("Experiment Comparison")

Print ("original model (128 neurons): ", test_accuracy)

Print ("modified model (64 neurons): ", test_accuracy)

ORIGINAL MODEL (128 NEURONS)					
Epoch 1/5	1688/1688	13s	7ms/step	accuracy: 0.9206	loss: 0.2796
Epoch 2/5	1688/1688	9s	5ms/step	accuracy: 0.9629	loss: 0.1244
Epoch 3/5	1688/1688	9s	5ms/step	accuracy: 0.9741	loss: 0.0858
Epoch 4/5	1688/1688	10s	6ms/step	accuracy: 0.9806	loss: 0.0651
Epoch 5/5	1688/1688	12s	7ms/step	accuracy: 0.9846	loss: 0.0495

EXPERIMENT COMPARISON

Original Model (128 neurons): 0.973800003528595
Modified Model (64 neurons): 0.9681000113487244